A CASE-STUDY BASED REPORT

ON

TRAFFIC FLOW PATTERN ANALYSIS USING K-MEANS AND DBSCAN

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

HAADI HASIL	**2520030297**
SATISH	**2520030302**
VEMULA GAUTAM	**2520030316**

Under the Esteemed guidance of

Dr. Ravindra

Asst.Professor, Dept. of CS&IT



(DEEMED TO BE UNIVERSITY)

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction

- Background
- Motivation for the Project
- Overview of the Work Carried Out
- Scope and Responsible-Use Statement

Problem Statement

- Case-Study Scenario
- Central Case-Study Question
- Formal Problem Definition
- Objectives that Guide the Study
- Assumptions and Limitations

Dataset and Exploratory Data Analysis

- About the Dataset
- Purpose of EDA
- Preparation Steps Actually Performed
- EDA Exploration - Implementation
- Summary of EDA Findings

Algorithm Selection and Implementation

Conclusion

Signature of the Team members

Abstract

Urban road networks generate continuous streams of sensor readings, yet most of this data carries no label telling us whether a road is flowing freely or is jammed. This case study applies unsupervised machine learning to discover traffic-flow patterns directly from three measurements recorded by road sensors: average speed (km/h), traffic volume (vehicles per interval) and lane occupancy (fraction of time a detector is covered). A 24-record sample collected across three road segments was cleaned, leaving 17 valid observations after seven malformed rows (missing or text values) were removed. Exploratory data analysis showed that volume and occupancy are almost perfectly correlated ($r = 0.99$) and both move opposite to speed ($r \approx -0.45$).

After standardization, two clustering algorithms were compared. K-Means with $K = 3$ recovered the free-flow, moderate and congested regimes (Silhouette 0.656, Davies-Bouldin 0.573), but it forced two abnormal, incident-like readings into ordinary clusters. DBSCAN (eps = 0.65, min_samples = 4) found the same three regimes with tighter separation (Silhouette 0.863, Davies-Bouldin 0.162 on clustered points) and, importantly, flagged both abnormal readings as noise. The full workflow is delivered as an interactive Streamlit application that supports CSV upload, parameter tuning, PCA visualization, cluster profiling and downloadable results. The study concludes that density-based clustering is better suited for traffic monitoring because it separates normal traffic states from rare anomalies.

Keywords: traffic flow, unsupervised learning, K-Means, DBSCAN, outlier detection, PCA, Streamlit

1. Introduction

1.1 Background

Traffic congestion is one of the most visible problems in growing cities. It increases travel time, fuel consumption and emissions, and it delays emergency services. Modern roads are instrumented with inductive loops, radar units and cameras that report speed, vehicle counts and occupancy every few minutes. Classical traffic-flow theory describes three related quantities: as more vehicles enter a road (volume and occupancy rise), average speed eventually falls, and beyond a critical density the road moves from free flow into congestion.

Machine learning lets us learn these regimes from data instead of fixing hand-picked thresholds. Because sensor data almost never comes with ground-truth labels such as "congested", unsupervised clustering is a natural fit: it groups similar observations together, and the analyst then interprets each group.

1.2 Motivation for the Project

- Traffic operators need a quick way to tell which road segments and time windows are free-flowing, moderately loaded or congested.
- Fixed thresholds (for example "speed below 30 km/h means congestion") do not adapt to different roads, speed limits or times of day.
- Real sensor feeds contain faulty and missing readings, as well as genuine anomalies such as accidents or sensor glitches, which must be separated from normal traffic.
- A simple, interactive tool lets non-programmers explore the data and see how algorithm settings change the result.

1.3 Overview of the Work Carried Out

The project implements an end-to-end pipeline in Python and Streamlit:

1. Data loading from a default sample file or a user-uploaded CSV.
2. Data cleaning: conversion to numeric values, removal of missing, text and physically impossible values (negative values or occupancy above 1).
3. Exploratory data analysis: preview, descriptive statistics, missing-value report, correlation matrix, histograms, box plots and scatter plots.
4. Feature standardization using StandardScaler.
5. Clustering with K-Means and DBSCAN, with sliders for K, eps and min_samples.
6. Two-dimensional visualization of clusters using Principal Component Analysis (PCA).
7. Cluster profiling (count, mean and median of each feature per cluster).
8. Evaluation using Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Score.
9. Download of clustered results and of the rows removed during cleaning.

Pipeline: CSV → Cleaning → EDA → Correlation → Scaling → K-Means / DBSCAN → Evaluation → Visualization → Download.

1.4 Scope and Responsible-Use Statement

The study is limited to clustering three numeric traffic features from a small sample dataset. It is intended for academic demonstration and for exploratory analysis by traffic analysts. The dataset contains only aggregated road-sensor values; it holds no personal, vehicle-registration or location-tracking information, so there are no privacy concerns.

Cluster labels such as "congested" are interpretations made by the analyst, not verified facts. Results from this tool should not be used on their own to issue penalties, change signal timings on live roads or make safety-critical decisions. Any operational use would require validation on a larger, real-world dataset and review by traffic engineers.

2. Problem Statement

2.1 Case-Study Scenario

A city traffic-management cell receives readings every five minutes from sensors on three road segments (Road-A, Road-B and Road-C). Each reading reports average speed, number of vehicles and lane occupancy. The cell has no labelled history telling it which readings represent normal flow and which represent congestion. Some sensor records also arrive broken: values are blank or contain text such as "abc", "unknown" or "high". Occasionally a reading looks unlike any normal traffic state, for example a very low speed with very few vehicles (a possible stalled vehicle or blocked lane) or a very high speed together with very high volume and occupancy (a likely sensor fault).

2.2 Central Case-Study Question

Can unsupervised clustering automatically discover meaningful traffic-flow states from speed, volume and occupancy readings, and which algorithm (K-Means or DBSCAN) better separates normal traffic patterns from abnormal, incident-like observations?

2.3 Formal Problem Definition

Given a dataset $D = \{x_1, x_2, \ldots, x_n\}$, where each observation $x_i = (\text{speed}_i, \text{volume}_i, \text{occupancy}_i) \in \mathbb{R}^3$, after removing invalid records and standardizing every feature to zero mean and unit variance, find a labelling function $f : D \rightarrow \{-1, 0, 1, \ldots, K-1\}$ that:

- assigns observations with similar traffic behaviour to the same cluster;
- maximizes separation between clusters (high Silhouette and Calinski-Harabasz scores, low Davies-Bouldin index); and
- optionally assigns the label −1 (noise) to observations that do not belong to any dense group.

No target variable is used for training. The illustrative_state column in the sample file is used only afterwards, to sanity-check the clusters.

2.4 Objectives that Guide the Study

1. Build a robust cleaning step that detects and removes missing, non-numeric and impossible values.
2. Understand feature distributions and relationships through EDA.
3. Apply K-Means and DBSCAN to the standardized features.
4. Interpret the resulting clusters as traffic regimes (free-flow, moderate, congested).
5. Detect outlier or incident-like readings.
6. Compare the two algorithms using internal validation metrics.
7. Package the workflow as an interactive, reusable Streamlit application.

2.5 Assumptions and Limitations

Assumptions

- Speed, volume and occupancy together are sufficient to characterize the traffic state.
- Occupancy is recorded as a fraction between 0 and 1; values outside this range are errors.
- Features measured on different scales should contribute equally, so standardization is applied.

Limitations

- The sample dataset is very small (24 rows, 17 valid) and covers about two hours on one day, so results demonstrate the method rather than prove general behaviour.
- Time of day, weather, road type and speed limits are not used as features.
- K-Means needs K chosen in advance and assumes roughly spherical clusters.
- DBSCAN is sensitive to eps and min_samples; the values used were tuned for this sample.
- DBSCAN metrics are computed only on non-noise points, which makes its scores look somewhat better than a like-for-like comparison would.

3. Dataset and Exploratory Data Analysis

3.1 About the Dataset

The dataset (data/traffic_sample.csv) contains 24 five-minute readings recorded on 6 October 2026 between 09:00 and 10:55 across three road segments. It was designed to include clear examples of each traffic regime, two abnormal readings and seven deliberately corrupted rows to test the cleaning step.

Column	Type	Description
timestamp	Date-time	Time of the reading (5-minute interval)
speed_kmh	Numeric	Average vehicle speed in km/h
traffic_volume	Numeric	Vehicles passing the sensor in the interval
occupancy	Numeric (0–1)	Fraction of time the detector is occupied
road_segment	Categorical	Road-A, Road-B or Road-C
illustrative_state	Categorical	Reference label for checking results only (not used in training)

Table 3.1: Dataset attributes

Reference state	Rows	Typical values
Free-flow	5	Speed 70–78 km/h, volume 20–28, occupancy 0.09–0.13
Moderate	5	Speed 45–52 km/h, volume 55–64, occupancy 0.30–0.37
Congested	5	Speed 16–23 km/h, volume 98–115, occupancy 0.68–0.78
Incident-like	2	5 km/h / 25 vehicles / 0.05, and 90 km/h / 115 vehicles / 0.85
Invalid rows	7	Blank or text values ("abc", "unknown", "high")
Total	24	

Table 3.2: Composition of the sample dataset

3.2 Purpose of EDA

- Check data quality: find missing, non-numeric and out-of-range values.
- Understand the range, centre and spread of each feature.
- Measure how features relate to each other, which affects clustering and PCA.
- Spot natural groups and possible outliers visually before running algorithms.
- Decide on preprocessing, such as whether scaling is required.

3.3 Preparation Steps Actually Performed

1. **Numeric conversion:** each feature column was converted using pd.to_numeric(errors="coerce"), which turns text such as "abc" into NaN.
2. **Missing-value removal:** any row with a NaN in speed, volume or occupancy was removed. Each feature had 3 unusable entries, affecting 7 rows in total.
3. **Range validation:** rows with negative values or occupancy greater than 1 were flagged as impossible (none were present in this sample).
4. **Audit trail:** removed rows were kept in a separate table and offered as a download (invalid_rows_removed.csv).
5. **Standardization:** the valid rows were transformed with StandardScaler so that speed (tens of km/h), volume (tens to hundreds) and occupancy (0–1) contribute equally to distances.

Time	speed_kmh	traffic_volume	occupancy	Reason removed
10:25	(blank)	70	0.40	Missing speed
10:30	abc	65	0.35	Text in speed
10:35	45	(blank)	0.30	Missing volume
10:40	50	unknown	0.32	Text in volume
10:45	48	60	(blank)	Missing occupancy
10:50	52	62	high	Text in occupancy
10:55	(blank)	(blank)	(blank)	All features missing

Table 3.3: Rows removed during cleaning (24 → 17 valid rows)

```
def clean_data(df):
    work = df.copy()
    for col in FEATURES:
        work[col] = pd.to_numeric(work[col], errors="coerce")
    invalid = work[FEATURES].isna().any(axis=1)
    bad, clean = work[invalid].copy(), work[~invalid].copy()
    impossible = ((clean["speed_kmh"] < 0) | (clean["traffic_volume"] < 0) |
                  (clean["occupancy"] < 0) | (clean["occupancy"] > 1))
    bad = pd.concat([bad, clean[impossible]], ignore_index=True)
    clean = clean[~impossible].copy()
    return clean.reset_index(drop=True), bad.reset_index(drop=True)
```

Code 3.1: Cleaning function used in app.py

3.4 EDA Exploration - Implementation

EDA was implemented in the first tab of the Streamlit app using pandas and Plotly. The descriptive statistics of the 17 valid rows are shown below.

Feature	Count	Mean	Std	Min	25%	Median	75%	Max
speed_kmh	17	47.29	26.29	5.00	21.00	48.00	72.00	90.00

traffic_volume	17	64.00	36.55	20.00	25.00	60.00	102.00	115.00
occupancy	17	0.40	0.29	0.05	0.12	0.34	0.70	0.85

Table 3.4: Descriptive statistics (valid rows)

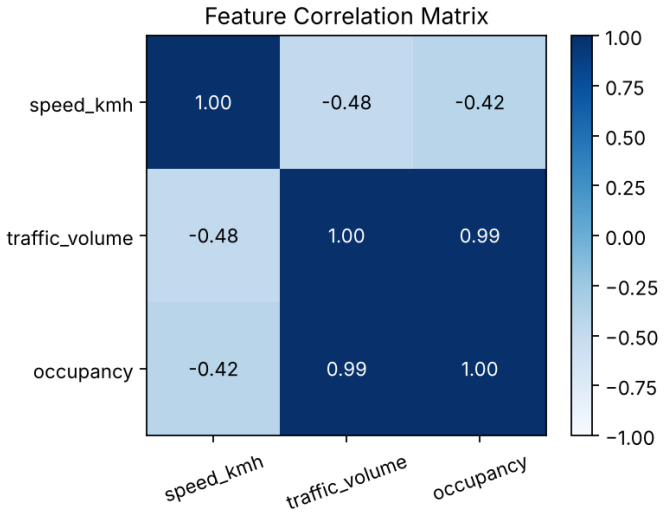


Figure 3.1: Correlation matrix of traffic features

Volume and occupancy are almost perfectly correlated (0.99): more vehicles means detectors are covered for longer. Speed has a moderate negative correlation with both volume (−0.48) and occupancy (−0.42). The negative relationship is weaker than expected because of the two incident-like readings, which break the usual speed–density trend.

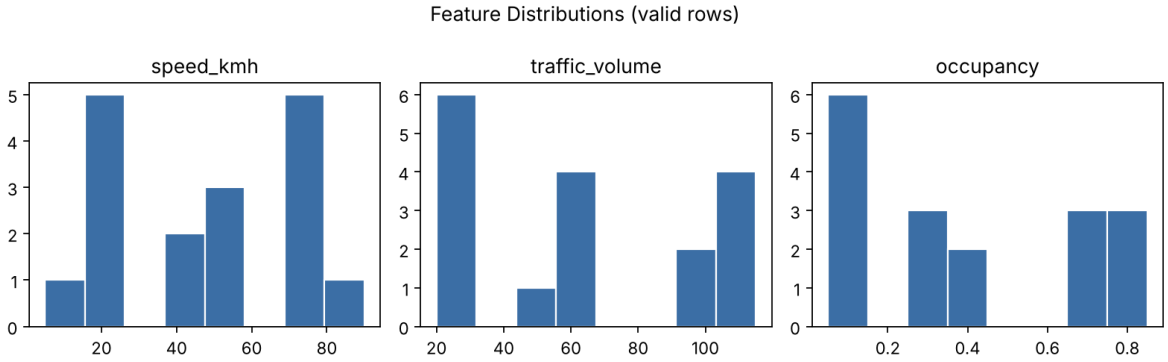


Figure 3.2: Distributions of speed, volume and occupancy

All three histograms are spread across separate ranges rather than forming a single bell curve, which hints that the data contains several distinct traffic regimes.

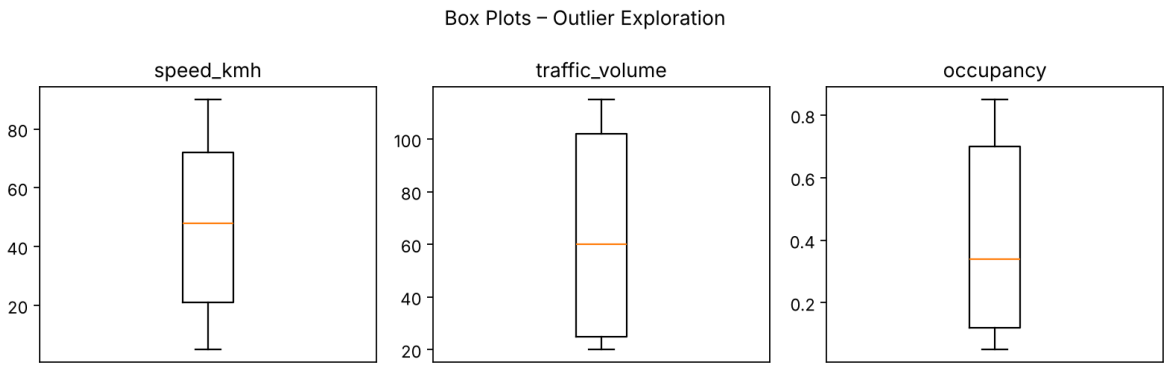


Figure 3.3: Box plots used for outlier exploration

Box plots show wide ranges but no points beyond the whiskers. This is an important finding: the two abnormal readings are not extreme in any single feature. They are abnormal only in their combination of values, so univariate outlier rules would miss them and a multivariate method is needed.

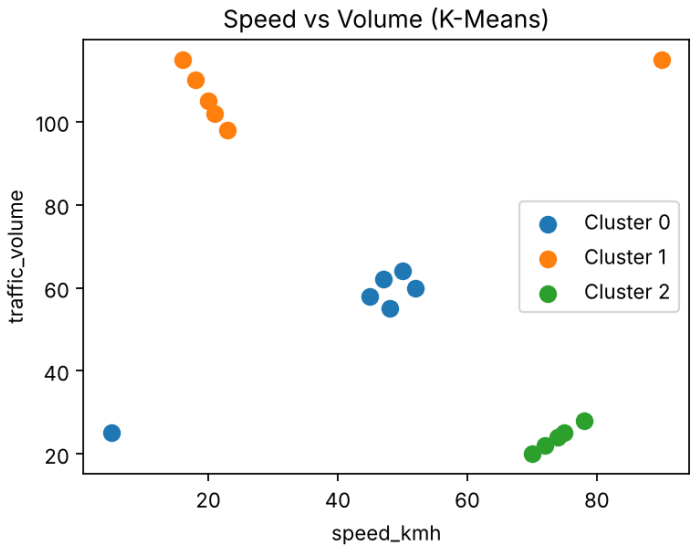


Figure 3.4: Speed versus volume scatter plot

```
st.plotly_chart(px.imshow(clean[features].corr(), text_auto=".2f"))
st.plotly_chart(px.histogram(clean, x=f, nbins=15))
st.plotly_chart(px.box(clean, y=features))
st.plotly_chart(px.scatter(clean, x=features[0], y=features[1],
color="road_segment"))
```

Code 3.2: EDA visualizations in the Streamlit app

3.5 Summary of EDA Findings

- 7 of 24 rows (29%) were unusable and were removed; 17 rows were used for modelling.

- Features are on very different scales (occupancy 0–1, volume up to 115), so standardization is required before distance-based clustering.
- Volume and occupancy carry almost the same information (r = 0.99); speed adds a separate dimension.
- The scatter plot shows three visually separate groups: high speed / low volume, medium / medium, and low speed / high volume.
- Two points sit away from these groups. They are not extreme in any single feature, so they can only be found by looking at all features together.

4. Algorithm Selection and Implementation

4.1 Why Unsupervised Clustering

Traffic sensors do not provide labels, so supervised classifiers cannot be trained. Clustering finds structure without labels. Two complementary algorithms were selected:

Aspect	K-Means	DBSCAN
Idea	Partitions data into K groups around centroids	Groups points in dense regions
Number of clusters	Must be given (K)	Found automatically
Cluster shape	Roughly spherical	Any shape
Outliers	Every point forced into a cluster	Labels sparse points as noise (−1)
Key parameters	K, random_state, n_init	eps, min_samples
Role in this study	Baseline regime discovery	Regime discovery + anomaly detection

Table 4.1: Comparison of the selected algorithms

4.2 K-Means Clustering

K-Means chooses K centroids and repeatedly assigns each point to the nearest centroid, then moves each centroid to the mean of its points, minimizing the within-cluster sum of squared distances. K = 3 was chosen to match the three expected traffic regimes, with random_state = 42 and n_init = 10 for reproducible results. A sweep of K from 2 to 6 was also run:

K	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Comment
2	0.600	0.621	25.8	Merges two regimes
3	0.656	0.573	25.0	Matches the three traffic regimes
4	0.708	0.360	43.8	Splits off one anomaly
5	0.762	0.125	309.8	Each anomaly becomes its own cluster
6	0.666	0.227	403.5	Starts splitting normal regimes

Table 4.2: Effect of K on K-Means quality

K = 5 scores best numerically only because it gives each of the two anomalies its own single-point cluster. That is a sign that the data contains outliers, which K-Means cannot label as such. K = 3 was kept because it gives the interpretable three-regime result.

4.3 DBSCAN Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) marks a point as a core point if at least min_samples points lie within distance eps of it. Clusters grow from connected core points, and points that cannot be reached from any core point are labelled −1 (noise). With

eps = 0.65 and min_samples = 4 on standardized data, the algorithm finds dense traffic regimes and isolates unusual readings without being told how many clusters to find.

4.4 PCA for Visualization

Principal Component Analysis projects the three standardized features onto two axes for plotting. PC1 explains 76.7% and PC2 23.2% of the variance (99.9% together), so the 2-D plots faithfully represent the 3-D data. PCA is used only for display; clustering runs on all selected features.

4.5 Implementation

The models are implemented with scikit-learn inside the Streamlit app. Sidebar sliders control K (2–8), eps (0.10–2.00) and min_samples (2–15), so the user can see the effect of each setting immediately.

```
X = StandardScaler().fit_transform(clean[features])

km_labels = KMeans(n_clusters=k, random_state=42, n_init=10).fit_predict(X)
db_labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X)

z = PCA(n_components=2, random_state=42).fit_transform(X)

def metrics(X, labels):
    mask = labels != -1                    # ignore DBSCAN noise
    if mask.sum() < 3 or len(np.unique(labels[mask])) < 2:
        return np.nan, np.nan, np.nan
    x, y = X[mask], labels[mask]
    return (silhouette_score(x, y), davies_bouldin_score(x, y),
            calinski_harabasz_score(x, y))
```

Code 4.1: Core modelling and evaluation code

Component	Tool / Library
Language	Python 3
User interface	Streamlit (≥ 1.35)
Data handling	pandas (≥ 2.0), NumPy (≥ 1.24)
Machine learning	scikit-learn (≥ 1.3): StandardScaler, KMeans, DBSCAN, PCA, metrics
Visualization	Plotly Express (≥ 5.18)
How to run	pip install -r requirements.txt → streamlit run app.py

Table 4.3: Technology stack

4.6 Results

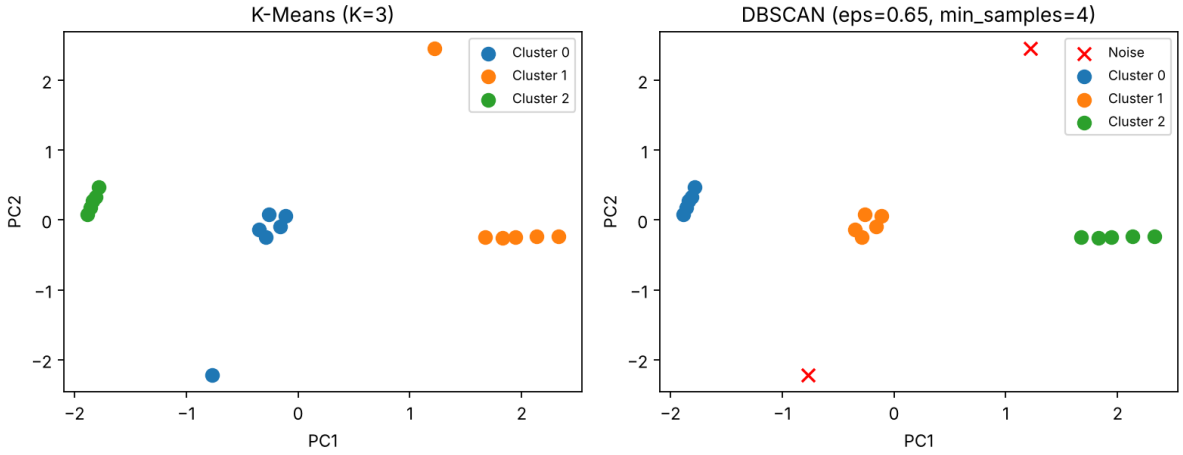


Figure 4.1: K-Means and DBSCAN clusters in PCA space (red × = DBSCAN noise)

K-Means cluster	Rows	Mean speed (km/h)	Mean volume	Mean occupancy	Interpretation
2	5	73.8	23.8	0.11	Free-flow
0	6	41.2	54.0	0.29	Moderate (+ 1 stalled-vehicle anomaly)
1	6	31.3	107.5	0.75	Congested (+ 1 sensor-fault anomaly)

Table 4.4: K-Means cluster profiles (K = 3)

DBSCAN label	Rows	Matches reference state
0	5	Free-flow (5 / 5)
1	5	Moderate (5 / 5)
2	5	Congested (5 / 5)
−1 (noise)	2	Incident-like (2 / 2)

Table 4.5: DBSCAN clusters (eps = 0.65, min_samples = 4)

Metric	K-Means (K = 3)	DBSCAN	Better
Silhouette Score (higher is better)	0.6559	0.8633*	DBSCAN
Davies-Bouldin Index (lower is better)	0.5729	0.1620*	DBSCAN
Calinski-Harabasz Score (higher is better)	24.99	459.18*	DBSCAN
Outliers detected	0	2	DBSCAN

*Table 4.6: Evaluation metrics (*computed on the 15 non-noise points)*

4.7 Discussion

Both algorithms agree on the 15 normal readings: each finds the same free-flow, moderate and congested groups. The difference lies in the two abnormal readings. K-Means placed the 5 km/h / 25-vehicle reading in the moderate cluster, pulling that cluster's average speed down to 41 km/h, and placed the 90 km/h / 115-vehicle reading in the congested cluster, raising its average speed to 31 km/h. These assignments distort the cluster profiles and hide the anomalies.

DBSCAN left both readings unassigned as noise, keeping the three regimes clean and giving the traffic cell a short list of readings to investigate. Its metrics are noticeably better, although part of the gain comes from evaluating only the non-noise points. The answer to the central case-study question is therefore: yes, clustering recovers meaningful traffic states, and DBSCAN is the better choice when anomaly detection matters, while K-Means remains a useful, simple baseline when the number of regimes is known.

5. Conclusion

This case study showed that unsupervised machine learning can turn unlabelled road-sensor readings into clear, interpretable traffic states. A careful cleaning step removed seven corrupted records, EDA revealed strong structure and the need for scaling, and both K-Means and DBSCAN recovered the free-flow, moderate and congested regimes. DBSCAN gave tighter clusters (Silhouette 0.86 versus 0.66) and, unlike K-Means, separated the two incident-like readings as outliers. The complete workflow is packaged as an interactive Streamlit application that lets users upload their own data, tune parameters and download results.

Key takeaways

- Data quality checks are essential: almost 30% of the raw sample was unusable.
- Standardization is required because traffic features use very different units.
- Anomalies in traffic data are often unusual combinations rather than extreme single values.
- Density-based clustering is better suited than centroid-based clustering for traffic monitoring.

Future scope

- Validate on a large public dataset (for example, PeMS or Metro Interstate Traffic Volume).
- Add time-of-day, day-of-week, weather and road-type features.
- Select eps automatically using a k-distance plot, and compare with HDBSCAN and Gaussian Mixture Models.
- Stream live sensor data and raise automatic alerts when DBSCAN flags noise points.
- Combine clustering with forecasting models (such as LSTM) to predict congestion before it occurs.

References

1. J. MacQueen, "Some methods for classification and analysis of multivariate observations," Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability, 1967.
2. M. Ester, H.-P. Kriegel, J. Sander and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," Proc. KDD, 1996.
3. P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," Journal of Computational and Applied Mathematics, 1987.
4. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR 12, 2011.
5. Streamlit documentation, https://docs.streamlit.io